

1) Why is it so hard to achieve even high training accuracy on CIFAR-10? Also, why is the test accuracy much lower than the train accuracy?

High training accuracy is hard because compared to MNIST, CIFAR-10 has much higher-dimensional input (MNIST: $28 \times 28 = 784$ gray pixels; CIFAR-10: $32 \times 32 \times 3 = 3072$ color values). The network has to carve out a decision region in a 3072-D space instead of 784-D, so the surface it needs to learn is much more complex. CIFAR also has way more variability & clutter. CIFAR images have backgrounds, different viewpoints, lighting, occlusions, etc. Two “airplanes” can look wildly different, and an airplane can share local patterns with “bird” or “ship”. A small FC net with width 450 and just 2 hidden layers simply doesn’t have enough effective capacity (given the optimizer and 20 epochs) to perfectly separate all 50k training images. Finally, you’re treating each pixel as an independent feature in a flat vector. The model has no notion of locality, edges, or translation invariance. Convnets build those assumptions in; your FC net has to learn them from scratch using only SGD, which is a much harder optimization problem. All of that means that even on the training set, gradient descent with these defaults doesn’t find a solution that classifies every training image correctly; it plateaus around ~60% train accuracy. Test accuracy is much lower than train accuracy because the model still has many parameters and no real regularization/augmentation, so it overfits patterns specific to the training set (backgrounds, colors, etc.) that don’t hold on new images, hurting generalization.

2) Why does the neural network make mistakes on the synthetic test set? Which model class would be better suited for this task?

The neural net only sees two blobs of points around (3,0) and (-3,0). It learns some nonlinear boundary that perfectly separates those blobs, but in the test rectangles its behavior is just an unconstrained extrapolation, so it misclassifies some points even though the true rule is just “which side of the y-axis are you on?”. A linear classifier (e.g., logistic regression or a linear SVM) is better suited here, because the true decision boundary is a vertical line; linear models are biased toward simple half-spaces and would generalize that rule perfectly from the blobs to the rectangles.